Verified Static Analyzer for a Toy Imperative Language

Fahim Islam

Static Analysis

- Static analysis:
  - Reasons about a program's behavior **without executing the program**.
  - Used in compilers, security tools, and IDEs.
  - Few are formally verified (unit testing used for 'confidence')
- **Goal**: detect errors before runtime.
  - Examples:
    - Divide by zero
    - Using an uninitialized variable
    - Null pointer dereferences

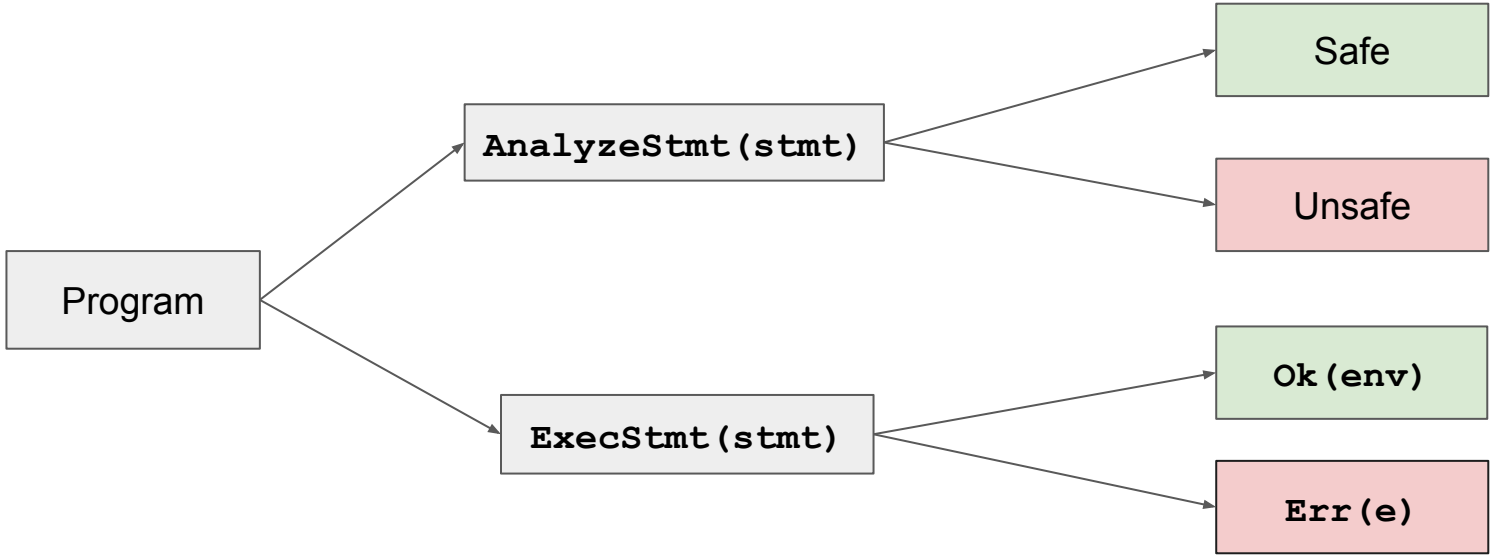
Abstract Interpretation

- Static analyzers reason about sets of possible values.
  - Instead of tracking real values
    - {x = 3, y = 0, …}
  - I will use abstract values
    - {x = NonZero, y = Zero, …}
- Benefit: Allows the analyzer to reason about many possible states with finite information.
- **`AbsState: map<string, AbsVal>`**. Stores abstract value for each variable: **`{Zero, NonZero, Unknown}`**

Toy Language Syntax

Expr	Stmt	RuntimeError
`Const(n: int)`	`Skip()`	`DivByZero`
`Var(x: string)`	`Assign(x: string, e: Expr)`	`UninitializedVar(x: string)`
`Add(e1: Expr, e2: Expr)`	`Seq(s1: Stmt, s2: Stmt)`	
`Sub(e1: Expr, e2: Expr)`	`If(cond: Expr, t: Stmt, f: Stmt)`	
`Mul(e1: Expr, e2: Expr)`		
`Div(e1: Expr, e2: Expr)`		
`Mod(e1: Expr, e2: Expr)`		

Pipelines: Analyzer vs. Runtime Execution



<u>Soundness Goal:</u>

If (`AnalyzeStmt(stmt)` = Safe) ⇒ (`ExecStmt(stmt)` ≠ `Err(e)`)

Safety Check: Expression

```
function CheckExpr(e: Expr, state: AbsState): bool⋯
{
    match e

    case Const(n) => true

    case Var(x) => x in state

    case Add(a,b) => CheckExpr(a, state) && CheckExpr(b, state)

    case Sub(a,b) => CheckExpr(a, state) && CheckExpr(b, state)

    case Mul(a,b) => CheckExpr(a, state) && CheckExpr(b, state)

    case Div(a,b) => CheckExpr(a, state) && CheckExpr(b, state) && AbsEval(b, state) == NonZero

    case Mod(a,b) => CheckExpr(a, state) && CheckExpr(b, state) && AbsEval(b, state) == NonZero
}
```

- Checks if an expression is statically safe.
- Recursively analyzes subexpressions to detect errors.

Example Test Cases

```
158 method TestCaseDivByZero(){
159     /*
160     v := 0;
161     v := 1 / v;
162     */
163     var s1 := Assign("v", Const(0));
164     var s2 := Assign("v", Div(Const(1), Var("v")));
165     var stmt := Seq(s1, s2);
166     var env := map[];
167     var state := map[];
168
169     var safe := AnalyzeStmt(stmt, state).0;
170     assert !safe;
171
172     match ExecStmt(stmt, env)
173     case Ok(env1) => assert false;
174     case Err(e1) => assert e1 == DivByZero;
175 }
```

```
177 method TestCasePruning(){
178     /*
179     if (1) then (x := 5) else (x := 1 / 0);
180     z := x;
181     */
182     var then_stmt := Assign("x", Const(5));
183     var else_stmt := Assign("x", Div(Const(1), Const(0)));
184     var s1 := If(Const(1), then_stmt, else_stmt);
185     var s2 := Assign("z", Var("x"));
186     var stmt := Seq(s1, s2);
187     var env := map[];
188     var state := map[];
189     var safe := AnalyzeStmt(stmt, state).0;
190     assert safe;
191     AnalyzeProgramSound(stmt);
192     match ExecStmt(stmt, env)
193     case Ok(env1) => assert env1["z"] == 5;
194     case Err(e1) => assert false;
195 }
```

Incompleteness Example

```
method TestCaseIncompleteness(){
    /*
    z := 2 + (-1);
    if (z) then (x := 5) else (x := 1 / 0)
    z := z + x;
    */
    var cond := Assign("z", Add(Const(2), Const(-1)));
    var then_stmt := Assign("x", Const(5));
    var else_stmt := Assign("x", Div(Const(1), Const(0)));
    var s1 := If(Var("z"), then_stmt, else_stmt);
    var s2 := Assign("z", Add(Var("z"), Var("x")));
    var stmt := Seq(cond, Seq(s1, s2));
    var env := map[];
    var state := map[];
    var safe := AnalyzeStmt(stmt, state).0;
    assert !safe;
    match ExecStmt(stmt, env)
    case Ok(env1) => assert env1["z"] == 6;
    case Err(e1) => assert false;
}
```

- **Runtime**
  - `2+(-1)= 1` → `cond` is true
- **Analyzer**
  - `NonZero + NonZero → Unknown`
- **Result**
  - Analyzer cannot prove the condition will only go one way
  - Marks unsafe due to error in else branch.
- Program is safe → Analyzer reports unsafe
  - (perhaps sound, but not complete)

Feedback on Future Direction

- **Which direction would be more worth pursuing?**
  - **Covering incompleteness cases and increasing precision**
    - Positive/Negative Domain flags
  - **Language Expansion**
    - Boolean Expressions/Comparison
    - Variable Declarations
    - While Loops
  - **Other suggestions?**

Questions?

Question #1: Pruning Validity

Q: How can I safely analyze only one branch?

- **`AbsEval(cond)== NonZero`**
  - Runtime will only execute **`then`** branch, and only then branch is analyzed.
- **`AbsEval(cond)== Zero`**
  - Runtime will only execute **`else`** branch, and only else branch is analyzed.
- **`AbsEval(cond)== Unknown`**
  - Runtime could execute either **`then`** or **`else`** branch, so both branches are analyzed.
  - The resulting state will be <u>intersection</u> of the resulting states from both branches.

Question #2: Conservative Nature of **`Add`** & **`Sub`**

Q: Why does: **`NonZero ± NonZero → Unknown`** instead of **`NonZero`**?

- The domain is intentionally coarse for simplicity, and for emphasizing ease of proving soundness increasing precision for completeness.
  - 1 + (-1) == 0.
- A finer domain (e.g. **`Positive`** & **`Negative`** flags) could improve precision.

Question #3: Soundness vs. Completeness

Q: What do soundness & completeness entail in the context of this project?

- **Soundness:** Given a program `p`, If the static analyzer indicates that `p` is safe → When `p` is executed, it will not result in either `DivByZero` or `UninitializedVar` errors.

- **Completeness:** Given a program `p`, If `p` doesn't result in either `DivByZero` or `UninitializedVar` errors when executed → Static analyzer will indicate that `p` is safe.

- This analyzer guarantees **Soundness.**

Question #4: `AbsState` Clarification

Q: Could the Analyzer track exact integer values?

- Yes, it could work due to the simplicity/restrictiveness of my language.
- However, this can become impractical when introducing new features:
  - user input
  - non-deterministic values
  - loops

Question #5: Integer Overflow

Q: Since the toy language allows for operations such as addition and multiplication with integers, is integer overflow a possibility? If so, how can it be dealt with/verified?

- Dafny's `int` type represents arbitrarily large mathematical integers. From a purely mathematical sense, these operations cannot overflow. Dafny's theorem prover can automatically handle these cases.
- Cool side note: Dafny has bounded integer types (`int8`, `int16`, etc.), and Dafny's verifier can check whether the results of operations will keep them within these bounds.

Question #6: Proof Structure

Q: What was the methodology for proving soundness?

- Proof by induction on the structure of statements.
- If all statements in a program are valid and consistency between AbsState and env is maintained (consistency invariant) → execution cannot produce a runtime error.

Soundness Goal: Codified

```
870     lemma AnalyzeProgramSound(p: Stmt)
871         requires AnalyzeProg(p)
872         ensures !IsErr(ExecStmt(p, map[]))
873     {
874         AnalyzeTrueExecValid(p, map[], map[]);
875     }
876
877     function AnalyzeProg(p: Stmt): bool
878     {
879         AnalyzeStmt(p, map[]).0
880     }
881 }
```